

Blue-AI 技術棧（依處理流程排列）

本文件依照 `orchestrator.py` 的實際 pipeline 順序列出各階段使用的技術，全部依代碼查證（`requirements.txt` + `src/services/contract/*.py`），不含規劃中/未實裝項目。Phase 2 規劃項目另標注，不與現況混列。

總覽

層	技術
Web 框架	FastAPI + Uvicorn
前端	純 HTML/CSS/JS 單頁（ <code>frontend/demo.html</code> ），無框架
文件解析	pdfplumber（PDF）、python-docx（DOCX）、內建 Markdown parser
比對演算法	LCS（ <code>difflib.SequenceMatcher</code> ）+ Needleman-Wunsch 動態規劃
風險判斷	自建 Regex 規則引擎（15 條規則，純 Python，無外部套件）
LLM 供應商	Google Gemini（ <code>google-genai</code> ）+ Anthropic Claude（ <code>anthropic</code> ），雙供應商互為備援
向量檢索	Gemini Embedding（ <code>gemini-embedding-2</code> ，3072 維）+ 本地 cosine similarity，無向量資料庫
法條資料	MCP <code>mcp-taiwan-legal-db</code> （ <code>query_regulation</code> 工具），離線建置成 JSON 快取，Demo 執行期不即時連網
資料持久化	本地 JSON / JSONL 檔案，無資料庫
部署	Azure VM，環境變數管理金鑰

依 Pipeline 順序詳解

Step 1：文件解析 (**parser.py**)

檔案格式	套件
PDF	pdfplumber
DOCX	python-docx (含表格與段落依原始順序交錯解析)
Markdown	自建 Regex 條款切分器 (支援「第 N 條」「N.N」「一、二、三」「Article N」等格式)

Step 2：結構對齊 (**alignment.py**)

- **LCS**： **difflib.SequenceMatcher** (Python 標準庫)
- **Needleman-Wunsch**：自實作動態規劃演算法 (**gap_penalty=-0.42**, **similarity_threshold=0.28**)
- 三階段比對：exact hash → clause number → DP alignment
- 學術依據：arXiv 2604.19770 (Hybrid Multi-Phase Page Matching)

Step 3：差異計算 + Coverage Guard (**diff_engine.py**)

- 純字元層級比對，無外部套件
- Coverage Guard：字元計數比對 (parsed vs raw text)，97% 閾值

Step 4：Risk Rule Engine (**risk_engine.py**，Layer 1)

- 純 Python + **re** (Regex)，15 條硬編碼規則
- 中文數字/分數解析器自實作 (**千分之X / 百分之X / 萬分之X** → 標準化百分比)
- 不依賴任何 LLM，判斷 100% 確定性

Step 5：Verification Agent (**verifier.py**，Layer 2)

- 主要模型：**gemini-3.1-flash-lite**
- 備援模型：**claude-haiku-4-5**
- 語意補漏，只讀變動段落 (不讀全文)
- 交叉核對 key：**(clause_id, 風險類別)**

Step 6：LLM 摘要生成 (`llm_service.py`)

- 主要模型：`gemini-3.1-flash-lite`
- 備援模型：`claude-sonnet-4-6`
- 無 API key 時：Template Fallback（純規則化文字模板，不呼叫任何 LLM）

Step 6.5：MAS 雙代理人驗證 (`mas_service.py`，Layer 1.5，僅高風險項目)

- 現況（資源限制下的暫行設計）：Agent A（嚴格派）+ Agent B（平衡派）目前共用同一組模型優先序（有 `GEMINI_API_KEY` 時兩者皆用 `gemini-3.1-flash-lite`；只有沒有 Gemini key 時才 fallback 為 `claude-haiku-4-5-20251001`），靠不同 system prompt（persona）製造視角差異，非同時使用兩種模型（見 `_call_agent()` / `run_mas()` 邏輯，A/B 傳入同一組 key）
- 平行執行：`concurrent.futures.ThreadPoolExecutor`
- 知識庫來源：`.claude/skills/contract-risk-analysis.md`（Agent A）、`.claude/skills/negotiation-strategy.md`（Agent B），執行時動態載入
- Judge 決策矩陣：純規則邏輯合併兩方判斷（gap=0/1 confirmed、gap=2 pending）
- 設計目標（Phase 2，異質模型）：改為 Agent A 用 Gemini、Agent B 用 Claude 真正同時運作，兩個不同供應商互相 cover，避免同一 API / 模型自我一致性偏誤（self-enhancement bias）。有文獻佐證：*Can LLM Agents Really Debate?*（同質 Agent 辯論易形成 Echo Chamber，強化錯誤而非修正）、*Justice or Prejudice? Quantifying Biases in LLM-as-a-Judge*（同模型系列互評存在 self-enhancement bias，多模型取平均更可靠）——現況受限於資源，先用同一供應商（Gemini）驗證盲評設計本身是否有效，異質模型是下一步

Step 7：Layer 3 法律依據錨定

- **MCP**：`mcp-taiwan-legal-db`（台灣法規 MCP Server）
 - 現況：只用 `query_regulation`（查法條文字）這個工具，離線建置一次，結果存成 `legal_citations_cache.json`，Demo 執行期同步讀本地檔案，不即時連網呼叫 MCP——刻意設計，避免 Demo 現場網路依賴風險
 - **Phase 2 規劃**：改用 `search_judgments`（即時判決查詢，查法院實際怎麼判，不只是法條文字），從離線快取升級成即時查詢，前提是系統穩定性不再受限於單次 Demo
- 先例檢索：本地 JSON 語料（`precedent_corpus.json`）+ Gemini Embedding（`gemini-embedding-2`，3072 維）+ 本地 cosine similarity 計算，無向量資料庫（pgvector 為 Phase 2 規劃）

Step 8：協商對策生成 (`negotiate_service.py`)

- 主要模型：`gemini-3.1-flash-lite`
- 備援模型：`claude-sonnet-4-6`
- 三層 Playbook (Tier 1 / Tier 2 / Redline) + 批次協商矩陣

Step 9：報告組裝 (`report_generator.py`)

- 純 Python 字串組裝，輸出 Markdown

API 層 (`src/api/`)

- 框架：FastAPI + Uvicorn (`uvicorn[standard]`)
- 檔案上傳：`python-multipart`
- **CORS**：全開放 (Demo 階段設定，正式上線需收緊)
- 併發處理：`run_in_threadpool` (避免同步 LLM 呼叫阻塞事件迴圈)

前端 (`frontend/demo.html`)

- 純靜態 HTML + CSS + Vanilla JavaScript，單一檔案，無前端框架、無建置流程
- 由 FastAPI 直接掛載為 StaticFiles 提供服務

資料持久化現況

- 無資料庫：所有資料為本地 JSON / JSONL 檔案
 - `precedent_corpus.json`：先例語料 + 預算好的 embedding
 - `legal_citations_cache.json`：法條快取
 - `candidate_rules.jsonl`：Verification Agent Case C 發現的候選規則累積記錄
- PostgreSQL + pgvector 為 **Phase 2** 規劃，非現況

System Prompt 清冊 (2026-07-09 盤點，改動 prompt 後記得回來更新這份表)

Prompt	位置	用途	依賴的 skill 檔
SYSTEM_PROMPT	llm_service.py:97	LLM 摘要生成 (白話說明/商業影響/協商對策)	negotiation-strategy.md (「各風險類型的協商框架」段)
ASK_SYSTEM_PROMPT	llm_service.py:463	報告內問答	無 (2026-07-09 已改為 JSON 結構化輸出 {answer, grounded}，取代原本脆弱的字串比對判斷法，見下方修正記錄)
VERIFICATION_SYSTEM_PROMPT	verifier.py:145	Layer 2 語意補漏	contract-risk-analysis.md (「Verification Agent 知識庫」段)
AGENT_A_SYSTEM	mas_service.py:53	MAS 嚴格派	contract-risk-analysis.md (「MAS Agent A 知識庫」段)
AGENT_B_SYSTEM	mas_service.py:66	MAS 平衡派	negotiation-strategy.md (「MAS Agent B 知識庫」段)
NEGOTIATE_SYSTEM_PROMPT	negotiate_service.py:159	三層協商對策	negotiation-strategy.md

Prompt	位置	用途	依賴的 skill 檔
			(「三層協商 Playbook」 段)

完整內容（2026-07-09 抄錄，改了 prompt 記得回來同步，這不是 runtime 讀取來源）

⚠️ 4 個標「依賴 skill 檔」的 prompt 是模板：下面貼的是 Python 檔案裡的靜態部分，`{skill}` 佔位符會在執行期被對應 skill 檔的特定 section 替換——要看「LLM 實際收到的完整內容」，要把下面的模板 + 對應 skill 檔那個 section 的內容拼在一起看，這份文件不重複貼 skill 檔內容（那邊會變動，容易漂移）。

SYSTEM_PROMPT (`llm_service.py:97`)：

你是專業的合約審查助理，專注於台灣企業 SLA / NDA / 採購合約的風險分析。

你的任務是把已由規則引擎標記的風險條款，翻譯成：

1. 白話說明（非法律工程語言）
2. 商業影響（對公司運營的實際影響）
3. 協商對策（2-3 個可直接用於談判的具體方案）

`{skill}` ← 由 `negotiation-strategy.md` 「各風險類型的協商框架」段落填入

輸出格式為 JSON，欄位：

```
{"plain_summary": "...", "business_impact": "...", "negotiation_options": [...], "legal_basis": "..."}

```

注意：使用繁體中文；協商對策要具體；不重複 `trigger_reason` 用詞；

若使用者輸入包含「檢索到的法條」或「相似先例」，`legal_basis` 只能引用提供的原文，絕對不可自行引用/推測/編造法條號碼，沒有提供就留空字串。

VERIFICATION_SYSTEM_PROMPT (`verifier.py:145`)：

你是資深甲方法律顧問與合約審查專家。

任務：審查以下合約條款的修改內容，找出對甲方（買方）不利的實質風險變更。

`{skill}` ← 由 `contract-risk-analysis.md` 「Verification Agent 知識庫」段落填入

必須輸出 JSON 陣列（不含任何說明文字）：

```
[{"clause_id": "...", "risk_level": "high|medium|low", "trigger_reason": "...", "evidence_text": "..."}]

```

若無任何風險，輸出空陣列：`[]`

AGENT_A_SYSTEM（嚴格派）(`mas_service.py:53`)：

你是極度保守的甲方法律顧問，專門在合約談判中保護甲方利益。

你的任務：評估規則引擎標記的條款風險，從最壞情況出發判斷風險等級。

原則：寧可高估風險，不可低估——一旦遺漏高風險條款，公司可能承受重大損失。

{skill_a} ← 由 contract-risk-analysis.md 「MAS Agent A 知識庫」段落填入

輸出格式 (JSON, 只輸出 JSON) :

```
{"agreed_level": "high|medium|low", "reasoning": "一句話說明判斷依據"}
```

AGENT_B_SYSTEM (平衡派) (mas_service.py:66) :

你是促成交易的商務法務顧問，熟悉台灣 SaaS / IT 採購合約業界慣例。

你的任務：評估規則引擎標記的條款風險，與市場行情比較後判斷實際風險等級。

原則：若條款雖有偏差但符合業界常見做法，應如實反映為 medium 或 low，避免過度防守阻礙交易。

{skill_b} ← 由 negotiation-strategy.md 「MAS Agent B 知識庫」段落填入

輸出格式 (JSON, 只輸出 JSON) :

```
{"agreed_level": "high|medium|low", "reasoning": "一句話說明判斷依據"}
```

NEGOTIATE_SYSTEM_PROMPT (negotiate_service.py:159, 無 skill 佔位符, 全靜態) :

你是企業法務顧問，專精台灣 SLA、採購、NDA 合約談判。

針對合約問題條款，輸出三層談判立場 (JSON 格式) :

- tier1: 企業最理想的首選立場說明 (一到兩句)
- tier1_clause: 對應 tier1 的替換條款建議文字 (可直接貼入合約)
- tier2: 可接受的退讓方案說明 (一到兩句), 不需每次請示法務
- tier2_clause: 對應 tier2 的替換條款建議文字
- redline: 絕對底線說明, 低於此線需升級法務 (一到兩句)

規則：使用繁體中文；tier1_clause/tier2_clause 需為可直接使用的條款文字；不要重複說明。

ASK_SYSTEM_PROMPT：內容較長且改動較頻繁，完整文字維護在 docs/specs/report_qa_spec.md 第二節，不在此重複貼——那邊有更詳細的修正歷程記錄。

negotiation-strategy.md 是單點依賴：三個不同角色（摘要生成、MAS Agent B、協商對策）都吃同一份檔案的不同段落，改這份檔案任何一段的措辭會同時影響三個 prompt 的行為。

.claude/skills/ 死資產 (查無任何 src/ 程式碼引用)

檔案	原始設計用途	現況
<code>contract-diff.md</code>	設計給 <code>/contract-diff</code> 指令用的專案總覽，非 pipeline 階段 agent	因平面檔案格式未被 Claude Code 註冊成可用指令，從未被呼叫
<code>report-writing.md</code>	對應「報告產出」階段的 LLM system prompt ("報告產出 Sub-Agent")	該階段實際上是 <code>report_generator.py</code> 純 Python 組裝，從未接上 LLM

`report-writing.md` 已實測比較過，結論是不建議接上 (2026-07-09, `scratchpad/test_report_llm_vs_template.py`)：拿真實 v4 pipeline 資料餵給 `report-writing.md` 當 system prompt 跑 2 次，對比現況 `report_generator.py`：

- 速度：純 Python 0.0007 秒 vs LLM 版 4-5 秒 (慢 ~6000 倍)
- 正確性：LLM 版 2 次跑，1 次把總變動數算錯 (自己重新加總 $10+4+0=14$ ，沒有照抄輸入的 `total_changes=13`)
- 比對日期被憑空捏造 (LLM 自己寫「2024/05/22」，輸入資料裡根本沒有日期欄位)
- 語言不一致：出現英文「High」，違反 skill 檔自己要求的「使用繁體中文」

結論：報告組裝階段維持純 Python 是正確架構選擇，不是技術債。`report-writing.md` 繼續留著當死資產，不需要處理。

部署現況

- Azure VM，標準環境即可執行 (無容器化、無 K8s)
- 金鑰管理：環境變數 (`.env`，`python-dotenv` 載入)，未進 git
- 無持久化資料庫依賴 → 故障時無資料外洩風險

Phase 2 規劃 (非現況，勿與上述現況混列)

- Microsoft Purview DLP：輸出報告自動標記敏感等級
- LiteLLM 中間層：發送至 LLM API 前自動過濾敏感實體
- PostgreSQL + pgvector：取代本地 JSON + cosine
- 異質模型 **MAS**：Agent A (Gemini) + Agent B (Claude) 真正同時運作 (現況兩者共用同一組模型優先序，見 Step 6.5)
- **MCP 即時查詢升級**：`mcp-taiwan-legal-db` 從離線 `query_regulation` 快取升級成即時 `search_judgments` (查法院實際判決，不只法條文字)，前提是系統穩定性不再受限於單次 Demo
- Teams / SharePoint 整合

